

# Effective dimensionality $d_{\text{eff}}(x, \tau)$ from MGD’s fitted $\theta_t$

Generated with Claude Code, from a code-reading session on 2026-09-23

## Purpose

Guth et al. define a local effective dimensionality  $d_{\text{eff}}(x, \tau)$  of a distribution’s support, at a point  $x$  and noise scale  $\tau$ , from the score of the variance-exploding (VE) noised distribution at  $x$ . This note derives  $d_{\text{eff}}$  directly in terms of MGD’s own fitted quantities –  $\theta_t, \varphi, \nabla\varphi$  – so it can be computed as pure post-processing of an already-completed MGD run, with no new sampling/fitting. Implementation: `codes/effective_dimension.py`. Notebook: `turbulence/effective_dimension_deff.ipynb`.

## 1 Guth’s $d_{\text{eff}}$ : recap

Let  $q_\tau$  be the law of  $y = x + \sqrt{\tau}\xi$ ,  $\xi \sim \mathcal{N}(0, I_d)$ ,  $x$  drawn from the data. Two equivalent definitions:

$$\text{energy form:} \quad d_{\text{eff}}(x, \tau) = d - \partial_s \mathbb{E}_y[U(y, \tau) \mid x], \quad s = \tfrac{1}{2} \log \tau, \quad (1)$$

$$\text{denoiser form:} \quad d_{\text{eff}}(x, \tau) = \frac{1}{\tau} \mathbb{E}_y[\|x - \mathbb{E}[x \mid y]\|^2 \mid x], \quad (2)$$

where  $U(\cdot, \tau) = -\log q_\tau(\cdot)$  and, by Tweedie’s formula,  $\mathbb{E}[x \mid y] = y + \tau \nabla_y \log q_\tau(y)$ . Both reduce to  $d$  for an isotropic Gaussian and to (less than)  $d$  when the data lies near a lower-dimensional manifold, at scales  $\tau$  finer than the manifold’s local curvature.

## 2 MGD’s schedule and sign convention

MGD’s Cos stochastic interpolant (`sde_routines.py`, `compute_interpolant`, `interpolant='Cos'` – the only schedule `forward_regularised` supports) is

$$I_t = \cos(\alpha_t) x_0 + \sin(\alpha_t) x_1, \quad \alpha_t = \frac{\pi t}{2}, \quad x_0 \sim \mathcal{N}(0, I_d), \quad x_1 = x \text{ (data)}. \quad (3)$$

**Sign convention.** This codebase fits  $\theta_t$  for  $p_\theta(x) \propto \exp(+\theta^\top \varphi(x))$  – the *opposite* of the MGD paper’s  $\exp(-\theta^\top \varphi(x))$ . This is audited directly against the code in `codes/utils_entropy.py` (“Sign convention audit”): the corrector step in `iteration_step_projection` is  $x_{k+1} = y_k + \nabla \varphi^\top \theta_{\text{raw}}$  (a *plus*), giving a continuous-time drift  $\sigma^2 \nabla \varphi^\top \theta_t$  that must equal  $\sigma^2 \nabla \log p_t$  for the transport to preserve  $p_t$ , so

$$\nabla_x \log p_t(x) = +\theta_t^\top \nabla \varphi(x). \quad (4)$$

Independently corroborated by  $dH_k = -\theta_k^\top \dot{m}_t$  (the entropy increment already computed in `iteration_step_projection`) and by  $\log Z_{\theta_1} \gtrsim H_{\text{bound}} + \theta_1^\top m_1$  (MGD Eq. 8/30 as implemented in `utils_entropy.log_Z_bound`). Every formula below is re-derived directly in this (+) convention – not by substituting  $\theta_{\text{paper}} = -\theta_{\text{code}}$  into the paper’s stated formulas.

### 3 The VE rescaling

Define  $y_t := I_t / \sin(\alpha_t) = x_1 + \cot(\alpha_t) x_0$ . Conditional on  $x_1 = x$ ,

$$y_t \mid x \sim \mathcal{N}(x, \tau(t) I_d), \quad \tau(t) = \cot^2(\alpha_t), \quad (5)$$

monotonically decreasing from  $\tau(0) = \infty$  to  $\tau(1) = 0$ . So MGD's Cos schedule already sweeps every VE noise scale, and  $t_{\text{reg}}$  (the grid  $\theta_t$  is fit on) IS a  $\tau$ -grid via Eq. 5.

Write  $q_\tau$  (Guth's VE marginal, density in  $y$ ) in terms of  $p_t$  (MGD's marginal, density in  $I_t$ ) by the change of variables  $I_t = \sin(\alpha_t) y$ :

$$q_\tau(y) = \sin^d(\alpha_t) p_t(\sin(\alpha_t) y). \quad (6)$$

Differentiating and using Eq. 4 (evaluated at  $I_t = \sin(\alpha_t) y$ ):

$$\nabla_y \log q_\tau(y) = \sin(\alpha_t) \nabla_x \log p_t(\sin(\alpha_t) y) = + \sin(\alpha_t) \theta_t^\top \nabla \varphi(\sin(\alpha_t) y). \quad (7)$$

The extra  $\sin(\alpha_t)$  factor is the chain-rule term the derivation asked to double-check – confirmed, with the (+) sign from Eq. 4 (a paper-convention derivation would carry a minus here).

*Practical note.*  $\sin(\alpha_t) y = \sin(\alpha_t) x + \cos(\alpha_t) \xi$  for  $y = x + \sqrt{\tau} \xi$  (since  $\sqrt{\tau} = \cot \alpha_t$ ) – literally `compute_interpolant`'s own Cos schedule with  $x$  as a single-point data endpoint and  $\xi$  as fresh noise. So every  $\varphi/\nabla\varphi$  evaluation below stays inside the numerical range the fit itself already explores; nothing is evaluated off-distribution in an uncontrolled way.

### 4 Denoiser form

Substituting Eq. 7 into Tweedie's formula,  $x - \mathbb{E}[x \mid y] = -\sqrt{\tau} \xi - \tau \nabla_y \log q_\tau(y)$ , so the  $1/\tau$  in Eq. 2 cancels analytically:

$$d_{\text{eff}}(x, \tau) = \mathbb{E}_\xi \left[ \left\| \xi + \sqrt{\tau} \sin(\alpha_t) \theta_t^\top \nabla \varphi(\sin(\alpha_t) y) \right\|^2 \right], \quad y = x + \sqrt{\tau} \xi. \quad (8)$$

No  $\log Z_t$ , no  $m_t$ , no normalisation constant – only  $\theta_t$  and  $\nabla\varphi$ . This is the primary estimator (`effective_dimension.denoiser_d_eff`).

### 5 Energy form

From  $U(y, \tau) = -\log q_\tau(y)$ ,

$$U(y, \tau) = -\theta_t^\top \varphi(\sin(\alpha_t) y) + \log Z_t - d \log \sin(\alpha_t). \quad (9)$$

The last term is analytic in  $t$ ; the first two are not, so Eq. 1's  $\partial_s$  is taken numerically, by finite-differencing  $\mathbb{E}_y[U(y, \tau) \mid x]$  directly across the  $t_{\text{reg}}$  grid (converted to  $s = \frac{1}{2} \log \tau(t)$ ), rather than splitting analytic/Monte-Carlo pieces (`effective_dimension.energy_d_eff`).

**$\log Z_t$ : a bound, not an exact value.** MGD's Prop. 4.3/Eq. 30 gives, in this code's convention,

$$\log Z_t \gtrsim H_{\text{bound}}(t) + \theta_t^\top m_t, \quad H_{\text{bound}}(t) = H(p_0) + \int_0^t dH_s ds, \quad (10)$$

generalised from MGD’s stated  $t = 1$  endpoint to any  $t$  (the integral is just truncated earlier). This is a *lower* bound on  $\log Z_t$ , so **the energy-form  $d_{\text{eff}}$  inherits that bound’s bias** – it is a secondary cross-check on the denoiser form, not an independent estimate.  $H_{\text{bound}}(t)$  is built by trapezoid-integrating the already-saved  $dH_t^{\text{bound}}$  (`effective_dimension.h_bound.at`);  $m_t$  is either the saved `aux_moments` (the run’s own realised target moments) or, if not pulled from the cluster yet, recomputed directly from the data and schedule,  $m_t = \mathbb{E}_Z[\varphi(\cos \alpha_t Z + \sin \alpha_t x_1)]$  – independent of any saved particle trajectory, so no SDE rerun either way (`effective_dimension.recompute_m.t`).

## 6 Dictionary

| Guth et al.                             | MGD (this codebase)                                                          |
|-----------------------------------------|------------------------------------------------------------------------------|
| VE noise scale $\tau$                   | $\tau(t) = \cot^2(\alpha_t)$ , $\alpha_t = \pi t/2$                          |
| score $\nabla_y \log q_\tau(y)$         | $+\sin(\alpha_t) \theta_t^\top \nabla \varphi(\sin(\alpha_t)y)$              |
| energy $U(y, \tau)$                     | $-\theta_t^\top \varphi(\sin(\alpha_t)y) + \log Z_t - d \log \sin(\alpha_t)$ |
| $\log Z_\tau$                           | $H_{\text{bound}}(t) + \theta_t^\top m_t$ ( <i>bound</i> , Eq. 10)           |
| noised sample $y = x + \sqrt{\tau} \xi$ | $\sin(\alpha_t) y = \text{compute\_interpolant}(x, \xi)$                     |
| per- $\tau$ fit                         | $\theta_t \equiv \text{Theta\_reg}[i]$ , grid $t_{\text{reg}}$               |

## 7 What’s used from a saved run

- `Theta_reg`, shape  $(n_t, r)$  – the regularised  $\theta_t$  trajectory (`codes/regularised.theta.py`); used exclusively (not the raw, finer  $\theta_t$ ) for a single self-consistent grid and to avoid a pre-fix off-by-one alignment issue present in some saved runs’ raw  $\theta_t/dH_t$  pairing.
- $t_{\text{reg}}$  – `Theta_reg`’s own coarse time grid; not saved for the  $d=256$  runs on disk as of this writing, so it is reconstructed offline (`reconstruct_t_reg`) by replaying `SDE.forward_regularised`’s block-accumulation and near-node-pruning exactly, from the saved fine grid  $t$  alone – verified to reproduce the on-disk `Theta_reg` length exactly.
- Each run’s `fitted_potentials/` –  $\varphi, \nabla \varphi$  with the SAME pruned active-statistic set the fit used (some potentials auto-prune during `fit()`; skipping this step silently mis-indexes every  $\theta$  coefficient).
- `dH_t_bound` + the fine grid  $t$  – integrated to  $H_{\text{bound}}(t)$  (Eq. 10), needed only by the energy form.
- `aux_moments` (`barphi_e`) if pulled from the cluster, else  $m_t$  recomputed from the run’s own  $x_1$  – needed only by the energy form.

## 8 Caveats

- The energy form differentiates a bound on a Monte-Carlo-noisy curve; expect it visibly noisier than the denoiser form at matched sample counts, and treat disagreements between the two as informative about the bound’s tightness rather than as a bug.
- $\theta_t$  becomes numerically extreme as  $t \rightarrow 1$  ( $\tau \rightarrow 0$ ): every seed inspected so far has one dominant, near-null-direction coefficient at the finest scale. This is harmless for the actual energy/score

(both stay smooth off-support – checked directly), but  $\theta_t^\top \varphi$  and  $\theta_t^\top \nabla \varphi$  must be computed in double precision, not the float32 the values are stored in.

- The finest available  $\tau$  is set by how close to  $t = 1$  the run’s schedule and `n_subsample` block reach, not by an arbitrary choice – see `codes/effective_dimension.py`’s `reconstruct_t_reg` for the exact grid actually available for a given run.